Lab 4 AI Exercise Reflections

SC2006 | HDB Resale Flat Search and Analysis System | Team ABCDE

Exercise 1 Recreating CpfGrantService

Our team used Claude Haiku through a Claude Code agent to recreate the deleted CpfGrantService. The agent received the tests, related code, grant configuration and class diagram, with instructions not to change the tests or hard-code grant amounts. Its first run failed because ProfileService called a method missing from the diagram. After reading that service and adding the method, it passed all 35 CPF tests and the full suite of 68 tests.

I expected it to do reasonably well because the tests described much of the required behaviour and the class diagram supplied the public method signatures. This made the task easier than implementing the class from requirements alone. Even so, I was impressed by how it connected the diagram with the surrounding code. The additional checks were more revealing: all four probes found differences despite the passing suite. This reduced my confidence in treating the existing tests as sufficient. They covered many normal cases and familiar boundaries, but missed combinations and configuration changes that could occur later.

The down-payment issue mattered most to me. Changing the default to 30% caused the AI version to enforce a 30% minimum, although the loan-to-value cap still allowed 25%. Users need accurate figures to trust our application. Missing explanations about approximate distances and buyer eligibility also needed restoring. Adding those sentences would be easy; checking that the application consistently provides them is the more important task.

I would keep the original implementation and selectively adopt useful improvements, such as the explicit error for a flat without a price. Before doing so, I would verify the change and add regression tests for the uncovered behaviours, especially the distinction between a default and a minimum. Next time, I would ask the agent to follow the project's existing conventions and seek clarification when an unclear business rule could change the result. If it struggled, I would provide the failing output and narrow the task before giving more structure. A second reviewer, possibly assisted by another AI, could help find omissions, but each finding would still need checking against the requirements. We did not test Cline, so we cannot conclude whether it would perform differently.

Exercise 2 Generating Class Diagrams and Identifying Design Differences

Our team used the same tool and model in a fresh session to generate class diagrams for UC-01 and UC-06 and compare the implementation with the Lab 3 design. Updating these diagrams was relevant to the final submission. In one round, the agent produced a PlantUML diagram and 23 reported differences. Checks confirmed that all 14 class names and 47 method names existed in the code, and four sampled differences were correct. A layout setting had to be added before the diagram rendered in the available environment.

The most useful output was the differences list. Keeping the design and implementation consistent is necessary, and manually finding every change across many files is time-consuming. However, confirming four findings does not establish that all 23 are correct. Each also needs interpretation: an omitted supporting class may indicate an outdated design, while leaving out getters or an internal thread pool may be an intentional simplification.

I was less convinced by the generated diagram itself. It was very wide, included too many accessors and was inconvenient to edit in our usual diagramming software. It also omitted the Strategy and Facade/Proxy classes. Our prompt's restriction to boundary, control and entity classes helps explain that omission, so we should have stated which design patterns needed to remain visible. For this task, I would draw the final diagram manually and use the generated version as a reference. Further exchanges about layout and formatting would probably cost more time than they saved.

Next time, I would use AI to check smaller changes as development progresses, rather than postpone the comparison until the end. I would specify the required notation, important relationships and level of detail, and ask it to flag uncertainty. Team members could then divide the findings for verification. That would make the review manageable and reduce the amount of documentation we have to revisit before submission.